

Risk Level: RISKY

32% safe

This code assumes trusted input and a stable in-memory user store, but it lacks validation, duplicate-user checks, and safe auth parsing. The hardcoded JWT secret and unhandled exceptions in `/dashboard` can lead to crashes or insecure deployments.

Fix These First

- 1

Calling `auth.split(' ')[1]` in `/dashboard` will throw `TypeError` if the `Authorization` header is missing, causing a 500 response or process instability depending on error handling setup.
- 2

`jwt.verify(token, SECRET)` will throw for expired, malformed, or forged tokens, and there is no `try/catch`, so invalid client input can trigger server errors.
- 3

A hardcoded secret (`mysecretkey`) is easy to leak, reuse, or guess, allowing attackers to forge valid tokens if they obtain it.

What the AI Assumed

- HIGH

The AI assumed `req.body.username` and `req.body.password` always exist and are valid strings in both `/register` and `/login`.
- HIGH

The AI assumed storing users in the in-memory `users` array is acceptable and persistent enough for authentication.
- HIGH

The AI assumed duplicate usernames are allowed or do not matter when `users.push({ username, password: hashed })` is called in `/register`.
- HIGH

The AI assumed `req.headers.authorization` is always present and formatted as `Bearer <token>` in `/dashboard`.
- HIGH

The AI assumed `jwt.verify(token, SECRET)` will not throw, or that crashing the request is acceptable.
- HIGH

The AI assumed the hardcoded `SECRET = "mysecretkey"` is secure enough for signing production JWTs.
- HIGH

The AI assumed there is no need for rate limiting or brute-force protection on `/login`.
- HIGH

The AI assumed usernames and passwords do not need length, format, or emptiness checks.

MEDIUM

The AI assumed bcrypt operations (``bcrypt.hash`` and ``bcrypt.compare``) will always succeed and do not need error handling.

MEDIUM

The AI assumed the JWT payload only needs ``username`` and that username values are trustworthy identifiers.

What Could Break

HIGH

Calling ``auth.split(' ')[1]`` in ``/dashboard`` will throw ``TypeError`` if the ``Authorization`` header is missing, causing a 500 response or process instability depending on error handling setup.

HIGH

``jwt.verify(token, SECRET)`` will throw for expired, malformed, or forged tokens, and there is no ``try/catch``, so invalid client input can trigger server errors.

HIGH

A hardcoded secret (``mysecretkey``) is easy to leak, reuse, or guess, allowing attackers to forge valid tokens if they obtain it.

HIGH

Registering the same ``username`` multiple times creates ambiguous login behavior because ``users.find(...)`` returns the first match, not necessarily the latest or intended account.

HIGH

Because ``users`` is an in-memory array, all accounts disappear on restart, do not work across multiple server instances, and can create inconsistent authentication behavior.

HIGH

If ``password`` is missing or not a string, ``bcrypt.hash(password, 10)`` and ``bcrypt.compare(password, user.password)`` may throw or behave unexpectedly.

HIGH

Without rate limiting, ``/login`` can be brute-forced and ``/register`` can be abused to consume memory by filling the ``users`` array.

MEDIUM

Without input validation, very large ``username`` or ``password`` values can waste CPU and memory, especially during bcrypt hashing.

MEDIUM

The server does not handle async route errors, so rejected promises in ``/register`` or ``/login`` can result in unhandled errors and unreliable responses.